ABNER MARTINEZ | PORTFOLIO

Security incident report and BSIMM assessment

Software Security | Full Sail University | January-February 2026

Academic coursework. The incident is a scenario assigned by the course: a classroom exercise, not a real incident. No live system was accessed, and the third-party firm, people and figures in the scenario are fictional.

Two documents: the incident report written for a non-technical audience, then a BSIMM-based assessment of the software security program behind it.

Redacted for publication: personal and scenario contact details, instructor name. Black boxes mark removed content.

Security Incident Report: FSO Application Vulnerability

Incident Summary

During a scheduled security assessment, a third-party firm identified critical weaknesses in the Full Sail Online (FSO) application. These weaknesses allow unauthorized individuals to view, change, or delete student and university records. Essentially, the "digital locks" on our student portal were found to be ineffective, potentially allowing any user to gain administrative power or tamper with the database.

Internal verification confirms these vulnerabilities have been present for approximately six months. The primary risk is a significant breach of student privacy, including the potential manipulation of grades and personal information. This poses a high risk to the university's reputation and our compliance with educational privacy laws. Immediate remediation is required to secure our data and restore the integrity of the FSO platform.

Date and Contact Information

- **Report Date:** January 12, 2026
- **Author:** Abner D. Martinez-Albizu
- **Author Contact:** [redacted]
- **Organization Security Officer:** Jack [redacted] CISO
- **Security Officer Contact:** [redacted]
- **Stakeholders Involved:** Johnny Hacksalot, Lead Penetration Tester, WeHackApps, LLC

Incident Description

Incident Description

The security incident was discovered on January 5, 2026, following a penetration test conducted by WeHackApps, LLC. The incident involves the Full Sail Online (FSO) application, which is hosted on our centralized web server cluster at the main campus data center.

Specific Impact Metrics:

- **Systems Affected:** 3 primary systems are affected, including the FSO Web Server, the Student Record Database, and the Authentication Server.

- **Users Affected:** Approximately 25,000 users are impacted, encompassing all currently enrolled students, faculty, and administrative staff.
- **Physical Locations:** One physical location (Main Campus Data Center).
- **Vulnerability Type:** The vulnerabilities identified include multiple forms of injection attacks and broken access control.
- **Discovery & Validation:** WeHackApps discovered the flaws; internal security teams validated these findings on January 7, 2026.
- **Incident Duration**: The vulnerabilities were introduced during a software update deployed six months ago, resulting in a window of exposure of 180 days.

Impact/Potential Impact

The risks associated with this incident are categorized as follows:

- **Loss of Data/Data Compromise:** Unauthorized access to student PII (Personally Identifiable Information) and academic records.
- **Damage to Integrity of Information:** The ability for users to manipulate backend data means that grades, financial records, or enrollment status could be altered without authorization.
- **Regulation Violation:** Potential violation of FERPA (Family Educational Rights and Privacy Act) due to the exposure of student records.
- **Public Relations Impact:** A breach of this nature could severely damage the university's reputation and trust with the student body.
- **Policy Violation:** Failure to maintain secure coding standards as outlined in the University Information Security Policy.

Sensitivity of Information Involved

Information compromised or at risk includes:

- [ ] **Public:** General course descriptions, campus maps, and public faculty bios.
- [X] **Internal Use Only:** Internal staff directories and university memos.
- [X] **Restricted or Confidential (Privacy Policy Violation):** Student PII (Social Security numbers, home addresses, phone numbers) and academic records (grades, transcripts, disciplinary actions).
- [X] **Unknown:** Any third-party integrated data not yet audited.

Mitigation

Upon validation of the vulnerabilities, the following targeted workstreams have been established:

1. Mitigations for Injection Attacks

- **Parameterized Queries**: The development team is migrating all database interactions to use prepared statements to ensure the database can distinguish between data and commands.
- **Input Validation:** Implementation of strict "allow-list" validation for all user input fields to ensure only expected data types are processed.

2. Mitigations for Broken Access Control

- **Logic Checks:** Adding code-level checks to every API endpoint to verify that the requesting user's ID matches the owner of the data being accessed.
- **Permission Audit**: Reviewing application logic to ensure users can only access data relevant to their specific permission levels.

Timeline

An emergency software patch must be pushed to the production environment within 30 days. A forensic audit of database logs will be conducted simultaneously to check for unauthorized manipulation over the last six months.

Notification

The following individuals and groups have been formally notified of this incident:

- **Garry Jones**, President of Full Sail University
- **Jack** [redacted] CISO of Full Sail University
- **Full Sail Security Team**

Sign-Off Information

Printed Name/Title: _____

Signature: _____

Security Incident Report: Application of BSIMM

Incident Summary

Following a recent security incident, Full Sail's information security leadership is performing an audit to move from a "feature-first" development approach to a mature Software Security Initiative (SSI). The audit identifies critical gaps: a lack of written security policies, no developer-specific training, and a disconnect between the security team and developers regarding infrastructure hardening and emergency response

---

GOVERNANCE

STRATEGY & METRICS

Full Sail's governance lacks formal strategy and specialized training, currently relying on individual interest.

- **Strategy & Metrics**

  SM1.1 (Publish process): Establish and publish a formal Software Security Initiative (SSI) that defines clear security goals, roles, and responsibilities tailored to the university's development lifecycle.

  SM2.7 (Evangelism): Designate a "Security Champion" to lead internal marketing efforts, build credibility through talks and roadmaps, and keep leadership informed on security challenges

---

COMPLIANCE & POLICY

While a specialist manages GLBA/FERPA compliance, there are no written procedures for faculty or developers to follow. There is a lack of understanding regarding PII handling, and training does not address the consequences of non-compliance.

- **CP1.1: Unify regulatory pressures**

  Action: Map GLBA and FERPA requirements into a single set of internal security controls. This ensures that a single development activity can satisfy multiple regulatory requirements at once.

- **CP1.2: Identify PII obligations**

  Action: Specifically audit all applications to identify where Personally Identifiable Information (PII) is handled and translate these legal obligations into technical software requirements.

- **CP1.3: Create policy**

  Action: Draft a mandatory software security policy that clearly defines what is "permitted" and "denied" for developers. This provides the compliance specialist with an auditable baseline.

- **CP2.5: Ensure executive awareness of compliance and privacy obligations**

  Action: Report on the results of the recent incident to executives in financial and legal terms to ensure they understand the stakes of non-compliance and continue to fund the SSI.

TRAINING

Faculty receive general security awareness training, but there is no specialized training for software developers or technical staff regarding the secure development lifecycle.

- **T1.1: Conduct software security awareness training**

  Action: Launch a mandatory awareness program for all developers, QA, and project managers that covers the "Why" and "How" of software security.

- **T1.7: Deliver on-demand individual training**

  Action: Provide developers with e-learning modules or a "Security Portal" containing resources they can access exactly when they encounter a security bug in their code.

- **T2.8: Create and use material specific to company history**

  Action: Build a training module specifically about the "Week 1" incident. Learning from "their own" mistakes makes the training more impactful for the Full Sail team.

- **T2.9: Deliver role-specific advanced curriculum**

  Action: Provide advanced tracks for specific technologies used at Full Sail, such as secure API development or cloud-native security for DevOps.

INTELLIGENCE

ATTACK MODELS (AM)

Full Sail uses high-level threat intelligence but does not tailor its defenses to its specific technology stack or software-level attack patterns.

- **AM1.5: Gather and use attack intelligence**

  Action: Feed external threat intelligence into the development process to create "Abuse Cases" that simulate how an attacker might target Full Sail's specific applications.

- **AM2.2: Create technology-specific attack patterns**

  Action: Develop a list of attack patterns specifically for the university's technology (e.g., if using Java or AWS), rather than relying on generic, high-level threats.

- **AM2.5: Maintain and use a top N possible attacks list**

  Action: Create a "Top 5" list of the most likely attacks against Full Sail's software (like SQLi or XSS) and use it to drive specific defensive design changes.

SECURITY FEATURES & DESIGN

Infrastructure hardening exists but is poorly communicated to developers. There is no regular exchange of best practices between the security and development teams.

- **SFD1.1: Build and publish security features**

  Action: Create a library of pre-approved security components (e.g., encryption or authentication modules) that developers can "plug in" to their projects without writing them from scratch.

- **SFD1.2: Engage SSG with architecture**

  Action: Establish a recurring meeting between the security team and lead developers to discuss architectural security early in the development of new features.

STANDARDS & REQUIREMENTS (SR)

There are few documented standards. While annual compliance reviews occur, vendor SLAs are missing any mention of software security requirements.

- **SR1.1: Create security standards**

  Action: Document specific, enforceable standards for coding (e.g., "no hard-coded credentials") that all developers must adhere to.

- **SR1.3: Translate compliance constraints to requirements**

  Action: Explicitly list GLBA/FERPA constraints as functional software requirements so developers know exactly what must be built to stay compliant.

- **SR2.2: Create a standards review process**

  Action: Implement an annual review where stakeholders from security and development collaborate to update standards based on the previous year's lessons.

- **SR2.5: Create SLA boilerplate**

  Action: Update all vendor contracts to include standard language requiring third-party software to meet Full Sail's security and vulnerability-fixing timelines.

CODE REVIEW

Full Sail lacks a formal code review process or standardized tools. Some developers use a free version of SonarQube, but it is not mandatory or tracked.

- **CR1.2: Have a formal code review process**

  Action: Make security-focused peer reviews a mandatory gate before code can be merged into the main codebase.

- **CR1.4: Use automated code review tools**

  Action: Deploy a standardized Static Analysis (SAST) tool across all development teams and integrate it into the CI/CD pipeline.

- **CR2.8: Use centralized defect reporting**

  Action: Ensure all security bugs are tracked in the university's central ticket system (e.g., Jira) to identify trends and improve training materials.

SECURITY TESTING (ST)

Quality Assurance (QA) is focused solely on feature functionality and performance. There is no adversarial or security-specific testing.

- **ST1.1: Ensure security requirements are tested**

  Action: Train QA testers to perform "Boundary Value Testing" probing what happens when unexpected or malicious data is entered into the application.

- **ST1.3: Drive tests with security requirements and security features**

  Action: QA must create specific test cases for security features (e.g., "verify account lock-out triggers after 5 failed attempts") to ensure they work as intended.

DEPLOYMENT

PENETRATION TESTING (PT)

External testing occurs annually for compliance, but there is no internal penetration testing capability or regular security scanning.

- **PT1.1: Use external penetration testers to find problems**

  Action: Continue annual external testing but hold a "De-brief" with the developers to turn the findings into learning opportunities.

- **PT1.3: Use penetration testing tools internally**

  Action: Train a member of the information security team to perform automated penetration scans quarterly to catch vulnerabilities between annual audits.

SOFTWARE ENVIRONMENT

Hardening requirements exist for general infrastructure but fail to cover the specific tools and environments used by the developers.

- **SE1.2: Ensure host and network security basics are in place**

  Action: Apply the university's OS and infrastructure hardening standards to the development and staging servers to ensure they are as secure as the production environment.

CONFIGURATION MANAGEMENT & VULNERABILITY MANAGEMENT

An Incident Response Plan exists for backups/recovery, but there is no process for emergency code changes or vulnerability remediation during an active attack.

- **CMVM1.1: Create or interface with incident response**

  Action: Update the IRP to define how the software development team will be engaged if application vulnerability is exploited.

- **CMVM2.1: Have emergency response**

  Action: Develop a "Fast Track" process to test and push emergency security patches to production without waiting for the normal weekly release cycle.

- **CMVM3.4: Operate a bug bounty program**

  Action: Establish a "Responsible Disclosure" policy on the university website to give ethical researchers a way to safely report vulnerabilities to Full Sail.